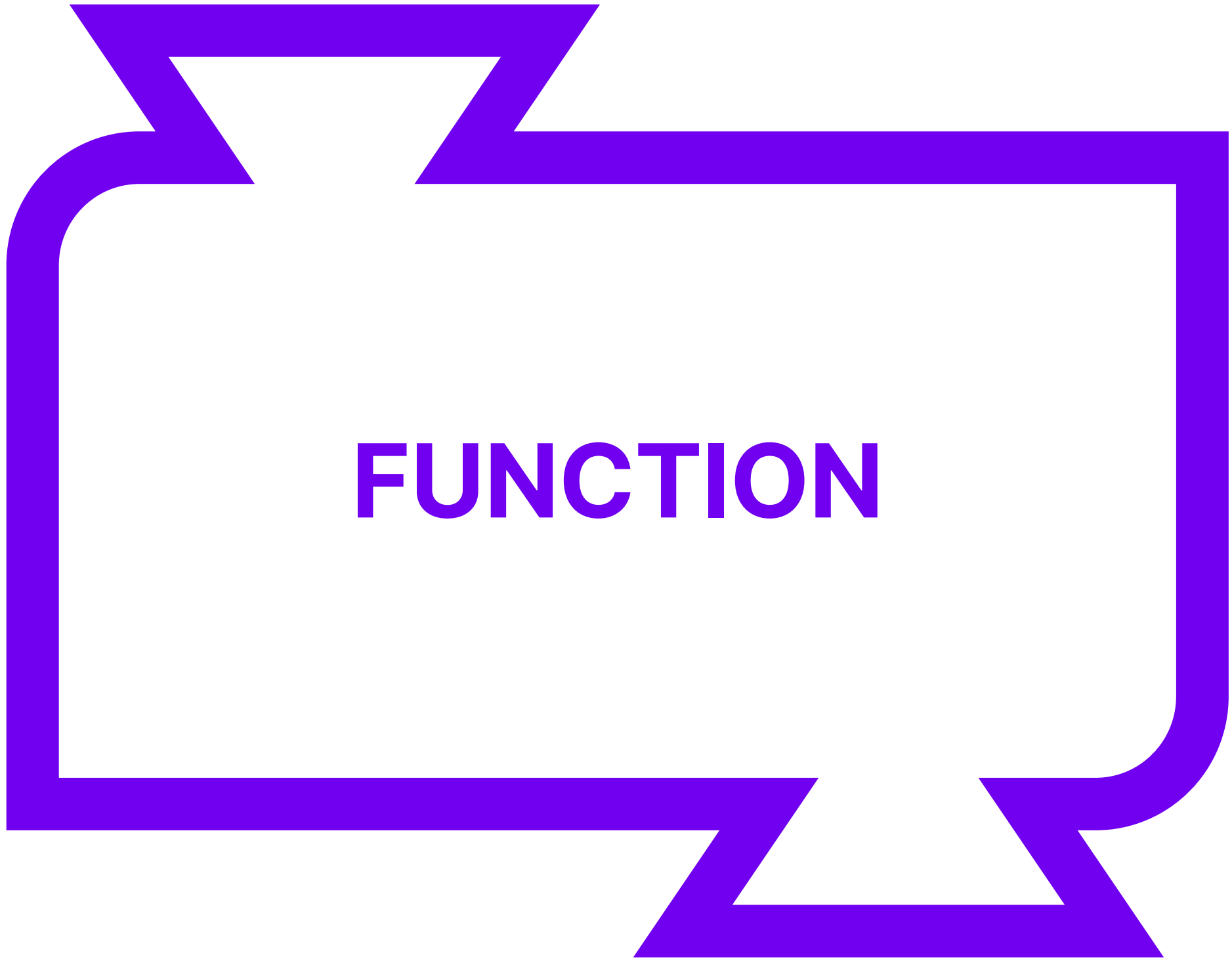


Python 기초 문법 2



함수란?

- 함수는 특정 작업을 수행하는 **코드 모음**
- 코드의 **재사용**이 쉽고, 가독성을 높여줌
- 코드를 더욱 효율적으로 관리할 수 있음

A large blue rectangular frame with a thick border. The corners of the frame are decorated with stylized, overlapping geometric shapes that resemble the 'Z' or 'N' patterns often found in programming syntax or design. The word 'FUNCTION' is centered within this frame.

FUNCTION



함수 정의하기

```
def 함수명(매개변수):  
    실행될 코드  
    return 반환값
```

- **def**로 함수의 이름을 정의하고, 전달 받을 **매개변수**를 지정함
 - 필요한 경우에만 매개변수를 사용함
- 들여쓰기를 사용하여 함수 내에서 실행될 코드를 저장함
- 함수에서 반환할 값이 있다면 **return**을 사용하여 반환할 수 있음



```
def 함수 이름(매개변수) :  
    함수에서 실행할 내용  
    return 반환값
```

```
함수 이름(인자)
```

선언한 함수의 이름을 호출하여 함수를 사용할 수 있음



매개변수와 인자

```
def 함수 이름(매개변수) :  
    함수에서 실행할 내용  
    return 반환값
```

함수 이름(인자)

```
def add_num(a, b) :  
    return a + b
```

add_num(1, 2)

- 매개변수는 함수에 전달된 값을 받는 변수
- 인자는 함수를 호출할 때 전달하는 값
- 함수를 정의할 때 사용하는지, 호출할 때 사용하는지에 따라 부르는 용어가 달라짐



매개변수의 사용

```
def add_num(a, b) :  
    print(f"{a} + {b} = {a + b}")  
  
add_num(a = 2, b = 3)          ## 2 + 3 = 5  
add_num(b = 2, a = 3)          ## 3 + 2 = 5
```

- 함수를 호출할 때, 매개변수를 지정하여 호출할 수 있음
- 지정한 매개변수에 순서에 상관 없이 값이 전달됨



```
def add_num(a, b = 3) :  
    print(f"{a} + {b} = {a + b}")  
  
add_num(2)                ## 2 + 3 = 5  
add_num(2, 4)              ## 2 + 4 = 6  
add_num(b = 4, a = 3)      ## 3 + 4 = 7
```

- 매개변수의 초기값을 지정하여 필요한 경우에만 바꿔줄 수 있음
- 초기값을 지정할 매개변수는 항상 가장 마지막에 위치해야 함
- 이 경우에도 매개변수를 지정하여 호출할 수 있음



매개변수의 *

```
def add_num_all(*args):  
    sum = 0  
    for n in args:  
        sum += n  
    return sum
```

arguments의 약자로
관례적으로 사용함

- 몇 개의 매개변수를 입력받는지 확실하지 않은 경우 * 기호를 사용할 수 있음
- * 기호를 사용해서 입력받은 경우, 해당 매개변수에 튜플로 저장됨



매개변수의 **

```
def print_info(**kwargs):  
    print(kwargs)
```

keyword arguments의
약자로 관례적으로 사용함

```
print_info(name = "Alice", age = 30, city = "New York")  
## {'name': 'Alice', 'age': 30, 'city': 'New York'}
```

- ** 기호가 붙은 매개변수는 **딕셔너리**로 생성됨
- 함수에 전달하는 인자를 딕셔너리 형태로 저장할 때 ** 기호를 사용할 수 있음



return문의 역할

```
def 함수명(매개변수):  
    실행될 코드  
    return 반환값
```



- return문은 함수의 실행을 **즉시 중단** 함
- 반환값을 가지고 함수를 호출한 지점으로 돌아감



return문 사용법

```
def 함수명(매개변수):  
    실행될 코드  
    return 반환값
```

- return문을 함수를 종료하고 싶은 위치에 작성함
- return 키워드 뒤에 반환할 값이나 표현식을 지정함
- 반환값이 없는 경우에는 return 키워드만 사용하는 것도 가능함



함수 사용 예시

```
def add_num(a, b) :  
    print(a + b)
```

```
add_num(1, 2)                ## 3
```

```
result = add_num(1, 2)       ## 3  
print(result)                ## None
```

```
def add_num(a, b) :  
    return a + b
```

```
add_num(1, 2)                ## 출력값 없음  
print(add_num(1, 2))         ## 3
```

```
result = add_num(1, 2)  
print(result)                ## 3
```

- 함수를 호출할 때마다 함수가 실행됨
- return 값이 없는 경우에는 **None**을 반환함



반환값의 개수?

```
def calculate(a, b):  
    return a + b, a - b
```

- 여러 개의 값을 한 번에 반환할 수 있을까?
- 오류가 발생하지는 않을까?



여러 개의 반환값

```
def calculate(a, b):  
    return a + b, a - b  
  
result = calculate(5, 2)  
print(result)          ## (7, 3)
```

- 여러 개의 반환값을 사용하는 것이 가능함
- 반환값이 여러 개인 경우에는 튜플로 저장됨
- 그렇다면 튜플을 분리해 따로 저장하고 싶다면?



여러 개의 반환값

```
def calculate(a, b):  
    return a + b, a - b  
  
plus, minus = calculate(5, 2)  
print(plus)          ## 7  
print(minus)         ## 3
```

- 두 변수에 반환값을 저장하게 되면, 각각의 반환값이 **다로 저장됨**
- 그렇다면, return문을 두 번 사용하는 경우에는?



여러 개의 반환값

```
def calculate(a, b) :  
    return a + b  
    print(a)  
    return a - b
```

```
result = calculate(5, 2)  
print(result)
```

1. 들여쓰기가 잘못됨
2. a와 b가 정의되지 않음

- 오류를 발생시킴
 - a와 b의 값을 구함으로 calculate 함수는 종료됨.
- return문 이후의 코드는 실행되지 않으므로 의미가 없음



변수의 스코프

- 변수가 접근할 수 있는 범위를 **변수의 스코프** 라고 함
- 파이썬에서는, 함수 내부와 외부에 따라 변수가 접근할 수 있는 범위가 달라짐

```
def 함수 이름(매개변수) :  
    함수에서 실행할 내용  
    return 반환값
```

함수 이름(인자)



```
num1 = 5
```

```
num2 = 2
```

```
def add_num(a, b) :  
    return a + b
```

```
result = add_num(num1, num2)  
print(result)
```

- 전역변수는 프로그램 전체에서 접근할 수 있는 변수
- 함수 외부에서 정의한 변수



```
num1 = 5
num2 = 2

def add_num(a, b):
    return a + b

result = add_num(num1, num2)
print(result)
```

- 지역변수는 **특정한 코드블록 내에서만 접근**할 수 있는 변수
- 함수 **내부**에서 정의하여 함수 내부에서만 유효한 변수
 - 함수가 호출될 때 생성되고 함수가 종료될 때 **소멸**됨



전역변수와 지역변수

```
num = 3
```

```
def add_num(a, b) :
```

```
    num = a + b
```

```
    return num
```

```
result = add_num(2, 4)
```

```
print("함수의 결과 :", result)
```

```
print("num의 값 :", num)
```

```
## 함수의 결과 : 6
```

```
## num의 값 : 3
```

같은 이름을 사용할 경우, 함수 내부에서는 지역변수를 우선시함



전역변수와 지역변수

```
def add_num(a, b) :  
    num = a + b  
    return num  
  
result = add_num(2, 4)  
print("함수의 결과 :", result)      ## 함수의 결과 : 6  
print("num의 값 :", num)           ## 오류 발생
```

- 함수 내에서 선언한 변수를 함수 외부에서 사용하는 경우 오류가 발생함
- 전역변수 num은 존재하지 않음



전역변수 global

```
num = 3
```

```
def add_num(a, b) :
```

```
    global num
```

```
    num = a + b
```

```
    return num
```

전역 변수 num을 사용한다는 의미

```
result = add_num(2, 4)
```

```
print("함수의 결과 :", result)
```

```
## 함수의 결과 : 6
```

```
print("num의 값 :", num)
```

```
## num의 값 : 6
```

global 키워드를 사용하면, 함수 내에서도 전역변수를 사용하고, 바꿀 수 있음



전역변수와 지역변수의 활용

전역변수	지역변수
프로그램 전체에서 접근할 수 있음	함수 내부에서만 유효함
함수 내부에서는 global 키워드를 이용해야 접근할 수 있음	동일한 이름의 변수라면 함수 내부에서 우선순위가 더 높음
프로그램 전체에서 공유되는 값이나 설정애 주로 사용됨	특정 함수 내에서만 필요한 임시적인 값이나 연산 결과 등에 주로 사용됨



람다 함수란?

```
def 함수 이름(매개변수):  
    return 반환값
```

```
lambda 매개변수 : 표현식
```

- 한 줄로 표현한 함수
- 이름이 없는 익명 함수



```
print((lambda x : x * 2)(3))    ## 6
```

```
func = lambda x : x * 2
```

```
print(func(3))                  ## 6
```

- **lambda** 키워드에 매개변수와 표현식을 이용하여 정의함
- 정의하며 **동시에 사용**할 수 있음
- 변수에 저장하여 사용할 수 있음



람다 함수 사용 주의사항

```
print((lambda x : x * 2)(3))    ## 6
```

```
func = lambda x : x * 2  
print(func(3))                ## 6
```

- 단순한 표현식 만 활용할 수 있음
- 복잡한 로직이나 여러 코드를 한 번에 작성하기는 어려움
- 간단한 연산이나, 함수를 인자로 전달 하는 경우에 사용함



람다 함수와 map

```
print(list(map(lambda x : x * 3, range(5))))  
## [0, 3, 6, 9, 12]
```

[0, 1, 2, 3, 4]

범위에 람다 함수를 적용함

- map 함수를 이용하여 모든 요소에 한 번에 람다 함수를 적용할 수 있음
- 함수를 인자로 전달하여 사용함